Next, we use docker to build our image for the web service with a single command:

```
$ docker compose build
```

This command will take a while. It will download an image. And most of the remaining time will be spent installing gems. If you run the same command again, it'll run quickly as nothing needs to be redone. If you change any file *other* than your `Gemfile`, the image will be updated quickly with the change. If you change the `Gemfile`, run `bundle update`, and then rerun `docker compose build`. This will take longer as it will rerun the `bundle install` step on a fresh image.

Next, we start the web container with a single command:

```
$ docker compose up
```

Normally, this command will be run with the `--detach` or `-d` option, which will run the containers in the background, but for now it's helpful to see the output.

At this point, your application is up and running! It can be accessed at http://localhost:3000/.

Using Console to Look at a Live Application

We've already created a large amount of functionality in our application's model classes. Of course, we created these to be used by our application's controllers—but we can also interact with them directly. The gateway to this world is the `rails console` script. We can launch it on our server with this:

```
$ docker compose exec web bin/rails console
Loading production environment (Rails 8.0.2)
work(prod)> p = Product.last
=> #<Product:0x0000ffff745c3f18 id: 3>
work(prod)> p.title
=> "Modern CSS with Tailwind"
work(prod)> p.price = 29.00
```